Studio Nx

백엔드 278 — 연결 서비스

2026-10-19

백엔드 278 — 연결 서비스 (services/connection_service.py)

항목	내용
파일	`studionx/backend/app/services/connection_service.py`
작업	S1-10a 클립·연결·패턴 API: 트랙 간 연결(Connect to Another)의 읽기, 저장, 끄기 서비스 (PRD 02 FR-2197)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 277 클립 서비스, 다음 279 패턴 서비스

1. 하는 일

한 트랙에서 다른 트랙으로 나가는 연결 한 건을 관리합니다. DB에 `uq_connections_from` 유니크가 있어서 `from_track_id` 하나에는 연결이 하나씩만 들어갑니다. 그래서 저장은 "보낸 값으로 통째로 맞추기", 지우기는 "연결 끄기" 두 가지뿐입니다.

```
get_connection(db, user, track_id)        트랙 확인 → 나가는 연결 조회 → 없으면 404
set_connection(db, user, track_id, data)  트랙 권한 확인 → 네 가지 검사 → 없으면 만들고 있으면 통째로 덮어씀
delete_connection(db, user, track_id)     트랙 권한 확인 → 나가는 연결 삭제 → 없으면 그냥 지나감
```

저장 전에 하는 검사는 네 가지입니다. 자기 자신으로는 보낼 수 없으므로 400으로 막고, 받는 트랙이 없는 트랙이거나 다른 프로젝트 것이면 404로 막습니다. 두 트랙이 서로를 가리키면 400으로 막고, 연결을 따라가다 다시 이 트랙에 닿는 고리가 생기면 400으로 막습니다. `target` 이 PLUGIN이면 받는 트랙의 이펙트 한 칸을 가리키어야 하며, 아니면 404로 막습니다.

2. 코드

```
def _outgoing(db: Session, from_track_id: int) -> TrackConnection | None:
    """이 트랙에서 나가는 연결 한 건 (없으면 None)."""
    return db.scalar(
        select(TrackConnection).where(TrackConnection.from_track_id == from_track_id)
    )


def _reaches(db: Session, start: int, target: int) -> bool:
    """start 에서 나가는 연결을 따라가다 target 에 닿는지 본다 (고리 검사).

    트랙마다 나가는 연결이 하나뿐이라 단순히 한 줄로 따라가면 된다. 지나간 트랙을
    다시 만나면 이미 고리이므로 멈춘다 (무한 반복 방지).
    """
    seen: set[int] = set()
    current: int | None = start
    while current is not None and current not in seen:
        if current == target:
            return True
        seen.add(current)
```

```
        nxt = _outgoing(db, current)
        current = nxt.to_track_id if nxt is not None else None
    return current == target
```

`set_connection` 은 저장의 본체이고, `delete_connection` 은 연결을 끕니다.

```
def set_connection(
    db: Session, user: User, track_id: int, data: TrackConnectionSetRequest
) -> TrackConnection:
    """연결을 보낸 값으로 통째로 맞춘다 (FR-2197). 새로 만들거나 지금 것을 고친다.

    잘못된 연결(자기 자신 · 서로 연결 · 고리 · 다른 프로젝트)은 400, 없는 받는 트랙이나
    이펙트는 404 로 막는다.
    """
    track = _require_editor_track(db, user, track_id)

    if data.to_track_id == track.track_id:
        raise AppError(ErrorCode.BAD_REQUEST, 400, "A track cannot connect to itself.")
    to_track = db.get(Track, data.to_track_id)
    if to_track is None or to_track.project_id != track.project_id:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Target track not found.")

    back = _outgoing(db, to_track.track_id)
    if back is not None and back.to_track_id == track.track_id:
        raise AppError(
            ErrorCode.BAD_REQUEST, 400, "Two tracks cannot be connected to each other."
        )
    # 받는 트랙에서 따라가다 이 트랙에 닿으면 고리가 된다. 지금 이 트랙에서 나가는
    # 연결은 곧 바뀌므로, 검사는 받는 트랙 쪽에서만 한다.
    if _reaches(db, to_track.track_id, track.track_id):
        raise AppError(ErrorCode.BAD_REQUEST, 400, "This connection would make a loop.")

    to_track_plugin_id = None
    if data.target == ConnectionTarget.PLUGIN and data.to_track_plugin_id is not None:
        plugin = db.get(TrackPlugin, data.to_track_plugin_id)
        if plugin is None or plugin.track_id != to_track.track_id:
            raise AppError(ErrorCode.NOT_FOUND, 404, "Target effect not found.")
        to_track_plugin_id = plugin.track_plugin_id

    connection = _outgoing(db, track.track_id)
    if connection is None:
        connection = TrackConnection(from_track_id=track.track_id)
        db.add(connection)
    connection.to_track_id = to_track.track_id
    connection.target = data.target
    connection.to_track_plugin_id = to_track_plugin_id
    connection.to_port = data.to_port
    connection.also_play = data.also_play
    connection.volume_db = data.volume_db
    connection.pan = data.pan
    connection.is_solo = data.is_solo
    db.commit()
    db.refresh(connection)
    return connection
```

```
def delete_connection(db: Session, user: User, track_id: int) -> None:
    """연결 끄기 (FR-2197). OWNER·EDITOR 만. 연결이 없어도 그냥 지나간다."""
    track = _require_editor_track(db, user, track_id)
    connection = _outgoing(db, track.track_id)
    if connection is not None:
        db.delete(connection)
        db.commit()
```

문법 풀이

식	뜻
`db.scalar(select(...))`	행이 하나뿐인 조회입니다. 있으면 그 행, 없으면 `None` 을 돌려줍니다
`_outgoing(db, track_id)`	유니크 때문에 이 트랙의 연결은 0개 또는 1개라 한 건으로 족합니다
`seen: set[int]` 와 `while current not in seen`	지나간 트랙 번호를 모아 두어 고리일 때 멈춥니다
`data.to_track_id == track.track_id`	자기 자신으로의 연결은 400으로 막습니다
`to_track.project_id != track.project_id`	남의 프로젝트 트랙은 404로 막습니다
`_reaches(db, to_track_id, from_track_id)`	받는 쪽에서 이 트랙까지 이어지는지 봅니다
`if connection is None:`	같은 트랙의 연결이 있으면 새 행을 만들지 않고 그 행을 고칩니다
`db.add(connection)`	유니크에 걸리지 않는 새 연결 행을 세션에 올립니다
`if connection is not None:`	없는 연결을 지울 때도 오류를 내지 않고 204로 지나갑니다

3. 왜 이렇게

선택	이유
저장을 PUT 으로 둠	한 트랙에서 나가는 연결은 DB 유니크(`uq_connections_from`) 때문에 하나뿐이라, 만들기와 고치기가 결국 같은 동작입니다
DELETE 가 연결을 끄는 역할	화면의 Connect 스위치를 끄는 값과 같습니다. 이미 꺼진 상태라도 204로 조용히 지나갑니다
고리 검사를 받는 트랙 쪽에서 수행	지금 이 트랙에서 나가는 연결은 곧 바뀌므로, 바뀐 뒤에도 고리가 없는지 받는 쪽에서 따라가 봅니다
지나간 트랙을 `seen` 에 기록	연결이 고리일 때 무한 루프에 빠지지 않게 합니다
서로 연결을 따로 검사	A에서 B로, B에서 A로 잇는 짧은 두 연결은 긴 따라가기 검사에서 놓칠 수 있습니다
이펙트는 받는 트랙의 것만 허용	`plugin.track_id != to_track.track_id` 이면 404로 막습니다
권한 확인을 `_require_editor_track` 에 모음	다른 트랙 API와 같은 규칙이라 남의 트랙은 404, 권한 없음은 403입니다

4. 검증

`tests/test_connection_routes.py` 열세 건이 모두 통과합니다. `test_put_creates_and_get_returns` 는 처음 저장과 읽기, `test_put_replaces_existing` 는 두 번째 PUT이 새 연결을 덮어쓰는지 확인합니다.

`test_put_rejects_self_connection`, `test_put_rejects_mutual_connection`, `test_put_rejects_loop` 은 세 가지 잘못된 연결이 400으로 막히는지, `test_put_to_other_project_is_404` 는 남의 프로젝트 404를, `test_delete_turns_connection_off` 는 DELETE 뒤 GET이 404가 되는지를 확인합니다. `test_viewer_cannot_write`,

`test_stranger_gets_404`, `test_needs_login` 은 권한과 로그인 규칙을 확인합니다. `pytest -q` 는 1083 passed, `ruff check .` 는 오류 없이 통과합니다.

5. 남은 일 (사용자)

확인할 것	내용
문서 검토	S1-10a 서비스와 라우터 문서(277 부터 281 까지)가 모두 끝난 뒤 한 번에 검토합니다
화면 연결 확인	Connect to Another 칸에서 트랙과 이펙트를 골라 저장하고 끄는 동작을 시퀀서에서 확인합니다